RFR Group 4 New Member Project

Contents

[Research & Goals](#)
[Notes/Justification](#)
[Design](#)

Goals & Research

What are your Goals?

- Create a circuit that monitors 16 temperature sensors
- Convert the data into digital signals and process said data
- Output over I2C Bus
- Components to Utilize:
    - Arduino Nano
    - DCDC Converter
    - Temperature sensor
    - Mutliplexer
    - Blinking Red LED
    - MOSFET

Why do we need these components?

- DCDC converter to step down to the adequate voltage needed for the Arduino and multiplexer
- Temperature sensor to actually get the data
- Multiplexer to be able to easily select which sensor we want to receive data from
- Arduino to be able to receive and convert the analog signal we get from the multiplexer
- Red LED to signify when a sensor is above 60 degrees C
- MOSFET in order to cycle Cn, Cn+2, Cn+4, etc., skipping adjacent cells.

Notes/Justification

Temperature sensor: [Sony-Murata-VTC5A-Li-ion-Battery-Module-With-Temperature-Sensor](#)

- The output voltage decreases as the temperature read by the sensor increases by around 5mV per increase in C
    - With 60 degrees C being represented as 1.51 V output
- When trying to read the temperature of multiple sensors, it should be done in two cycles, skipping over adjacent battery cell sensors
- Sensor characteristics that are important to take into consideration:
    - Vmin = Vcc - Vout, with a minimum of 10mV
    - Vcc should be >2.5V
    - Forward current = (Vcc - Vout)/R, with a minimum of 0.4 and a maximum of 15 mA.

Multiplexer: CD74HC4067

- When looking at the operating conditions, several things stood out, particularly the recommended operating conditions:
    - Vcc min of 2 and max of 6 V
    - Analog switch max voltage of Vcc
    - And a max input rise and fall time of 400 to 1000 ns, depending on a range of 2.5 to 6v

Arduino: Arduino Nano

- Vcc should be 5V
- Contains 14 digital and 8 analog pins
- Cannot draw more than 50mA from the board

DCDC converter: TSR 1-2450

- From the specifications above, we decided that a good DCDC converter is the TSR 1-2450 for several reasons
- Great range input to output
    - To output 5 V DC, you can input a range from 6.5-36 VDC
    - With 94% efficiency
- Allows a max output current of 1A
    - Meeting all of our requirements and more.
- Price: around $6.28 per converter, making it not a super expensive part
- Contains short-circuit protection

We still need a way to signify if the temperature goes above the threshold, via an LED
Sample LED (Unsure if we need it in the doc but included): RED LED

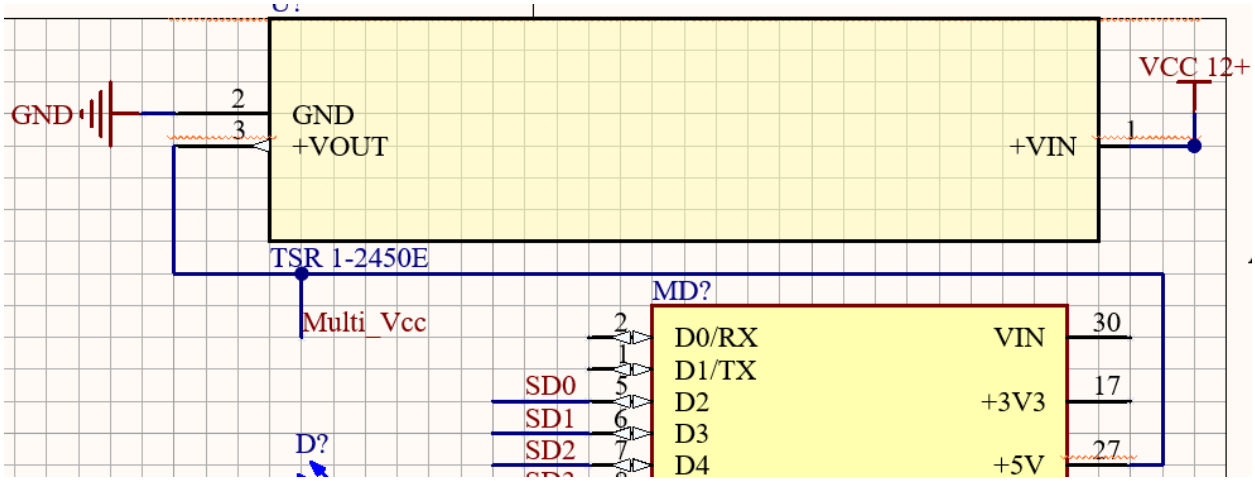
- Typical forward voltage of 1.9V
- Forward current of 10mA
- Therefore, needs a 300 Ω resistor via R = (Vcc-Vf)/(If)

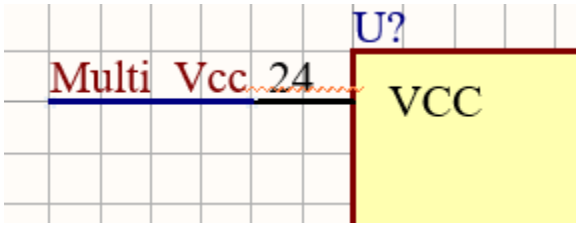
MOSFET: 2N7000

- Max gate-to-source voltage is 20V
- Max drain-to-source voltage is 60V
- 200mA max to drain pin
    - Easiest way: including a 100omh resistor

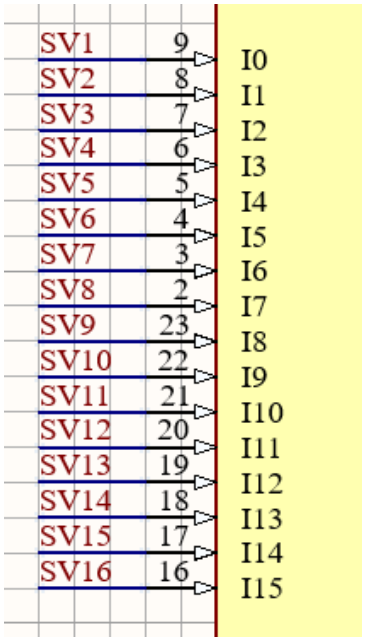
Design

In order to step down the voltage from 12V to 5V, that is the operating voltage of the Arduino and multiplexer, we used the DCDC converter and decided that the Vin pin for the Arduino is pin 27 and not 30 due to pin 30 requiring 7-12V.





To communicate with all 16 temperature sensors (Sv) and get the data we want when we want, we decided to connect the I0-I15 pins of the multiplexer to the sensor cathode so we can measure the sensor Vout, which corresponds to temperature.



Now in order to meet the requirement of measuring the temperature of the sensors Cn, Cn+2, Cn+4, etc we decided to make two sides (at least on the schematic), corresponding to one for the even sensors and one for the odd sensors; odd is left, even is right. We also decided to try to make a close replica of the Figure 6 diagram in the temperature sensor data sheet; however, because we couldn't vertically stack 16, we had to improvise by splitting it as you see.

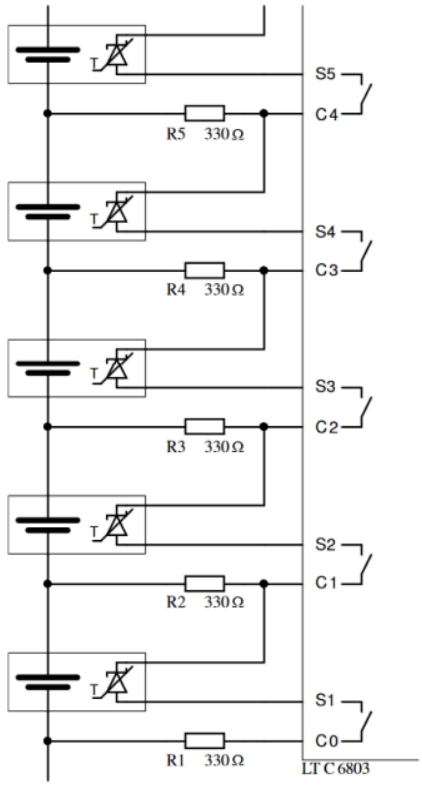
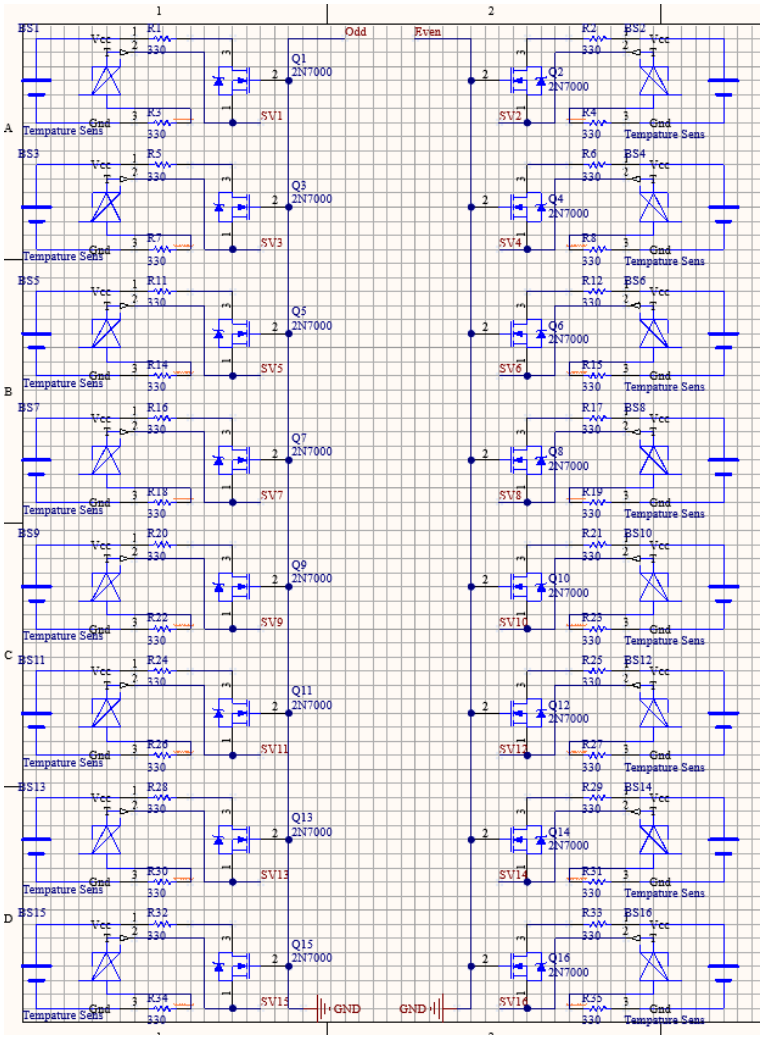
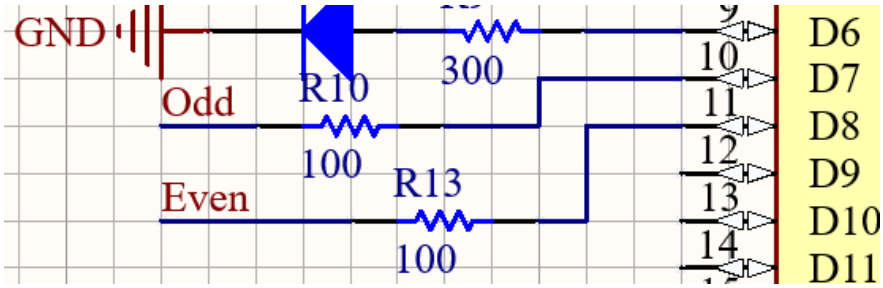


Figure 6: Suggested stack temperature acquisition circuit (simplified).

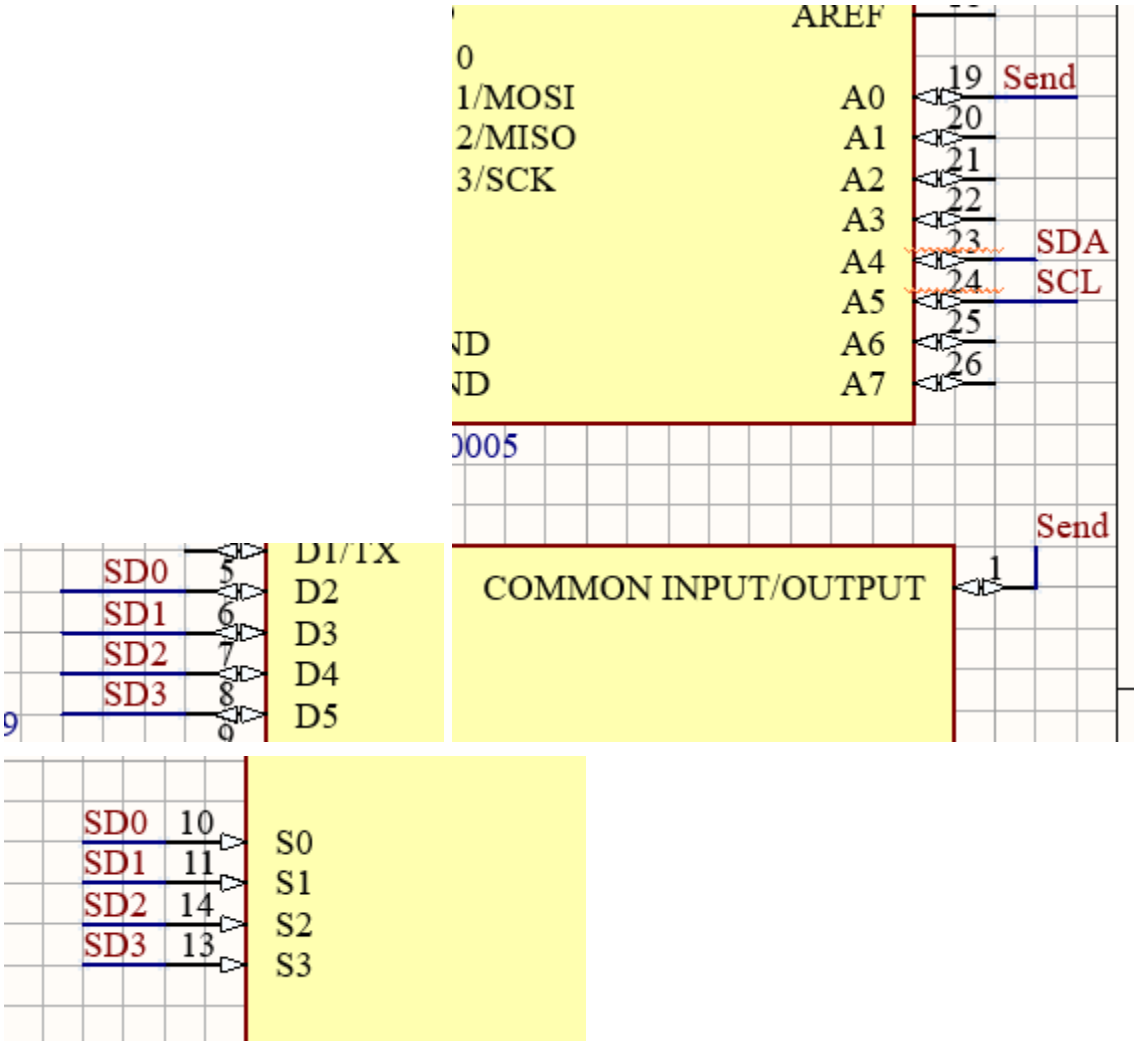


The only difference between the two sides is that the drain gate of the MOSFETs on the odd and even sides is powered by the Arduino's D7 and D8 pins, respectively. And in order to meet the requirement of 200mA or less directed to the drain gates of the MOSFETs, there is a 100 ohm resistor from pins D7 and D8



Now we still need a way for the Arduino and Multiplexer to communicate; the clearest and easiest way we found was for pins D2-D5 of the Arduino to connect to pins S0-S3 of the multiplexer. This works because the I0-I15 pins correspond to the digital states of S0-S3 on the

multiplexer, which selects which sensor's data we want and sends it back to the Arduino via an analog pin (C I/O to A0). Then we would send that data via I2C from the Arduino to the receiving device through pin A4, as it's the SDA pin.



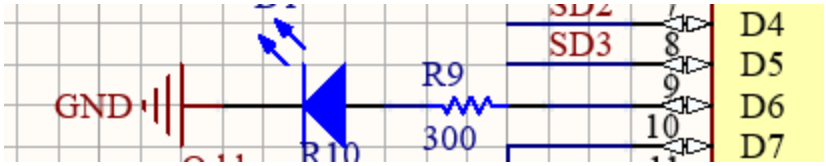
```
// Digital pins to dictate what sensor the multiplexer should get info from
int dig[4]={2,3,4,5};
// Pin that controls blinking LED
int D6 = 6;
// Pins that control which sensors we want to have power and get data from:
int odd = 7;
int even = 8;
// decides if the LED is lit or not
bool LED = false;
// Helps store the highest sensor data.
double maxTemp = 5;
double intTemp = 0;
void setup() {
  pinMode(dig[0], OUTPUT);
  pinMode(dig[1], OUTPUT);
  pinMode(dig[2], OUTPUT);
  pinMode(dig[3], OUTPUT);
  // Pin that receives data from mutliplexer C I/O pin
  pinMode(A0, INPUT);
  pinMode(D6, OUTPUT);
  // Pins for power to get data from set
  pinMode(odd, OUTPUT);
  pinMode(even, OUTPUT);
  // Initialized I2C
  Wire.begin();
}
```

```
  digitalWrite(odd, HIGH);
 // reads through all odd sensors first (since according to datasheet for the sensor, we should illiterate Cn, Cn+2, Cn+4, etc)
 for(int i = 1; i<16; i+=2){
   digitalWrite(dig[0], bitRead(i, 0));
   digitalWrite(dig[1], bitRead(i, 1));
   digitalWrite(dig[2], bitRead(i, 2));
   digitalWrite(dig[3], bitRead(i, 3));
   // Sends the read signal to be converted to see if tempature is above threashold, and if it is, the LED STAYS LIT till the next pas
   intTemp = convert(analogRead(A0));
   if(intTemp < maxTemp){
     maxTemp = intTemp;
   }
```

```
// method to return the voltage red as a DC signal using the threashold of 1.51V being 60C
double convert(double read){
  // calculates Vin
  double vin = (read* 5.0)/(1023.0);
  if (vin<=1.51){
    LED = true;
  }
  return vin;
}
```

```
digitalWrite(even, LOW);
// Sends the highest tempature found via I2C,  thanks to well documented adruino code: https://docs.arduino.cc/learn/communication/wire/
Wire.beginTransmission();
Wire.print("Highest Tempature found: ");
Wire.print(VtoT(maxTemp));
Wire.print("C");
Wire.endTransmission();
```

Lastly, using an ADC converter, D6 can power a permanent off state or a cycling/blinking state if the temperature of one of the sensors is equal to or greater than 60 degrees C.




```
// For the LED to blink when a sensor has been detected over the max threashold
for(int k = 0; k <10 && LED; k++){
    digitalWrite(D6, LED);
    delay(40);
    digitalWrite(D6, !LED);
    delay(40);
}
```